Prompt Engineering: The Prompt Ladder

Systematic Layer-by-Layer Optimization in Backend Engineering | Implementation Report

Author: Deepak	**Track:** Backend & Distributed Systems	**Runtime:** Python 3.11 / FastAPI / asyncpg / Redis
GitHub: github.com/deepak007679/prompt-ladder-assignment	**Runs Total:** 6 (Baseline + 5 Layers)	**Date:** October 2026

1. Executive Rationale & Methodology

The gap between a lazy prompt and an engineered one is the cheapest performance upgrade in AI, yet most engineers never witness it because they modify multiple variables simultaneously. This project adheres to strict experimental discipline: **one isolated layer is introduced per version**, attacking the primary weakness identified in the preceding output. Furthermore, by closely inspecting behavioral deltas rather than superficial changes, we captured an authentic **regression failure in Run 3**, demonstrating how misguided negative constraints degrade production software.

2. The Prompt Ladder Matrix (Overview of 6 Runs)

Run	Layer Added	Prompt Focus	What Actually Improved in Output	Remaining Flaw / Regression
Run 0	Baseline	"Write backend code"	None (Baseline starting point)	Guessed Node.js, generic items array, 10 scoping questions.
Run 1	Clearer Goal	User auth, password reset, session tokens	Stopped asking 10 scoping questions; focused 100% on auth domain.	Guessed synchronous Flask with direct SQLite connections and raw SQL.
Run 2	Real Context	Python 3.11, FastAPI, async SQLAlchemy 2.0, Redis	Adopted async/await, asyncpg, DeclarativeBase Mapped models, Redis pools.	350-line monolithic single-file code dump mixing all layers.
Run 3	**Constraints** (Regression)	Stdlib only, <60 lines, no comments	Output compressed to 40 lines; zero rambling prose.	**CRITICAL BREAKDOWN:** Unsalted SHA-256, timing attacks, stripped DB & Redis.
Run 4	Specified Output Format	3 modular files: schemas, core/security, api/v1	Clean architectural boundaries; separated validation, crypto, and routing.	Zero automated verification that tokens, expiry, or Redis revocation work.
Run 5	Verification Requirements	Async pytest suite: happy path, 401, expiry, Redis block	Generated executable pytest tests; forced router to use mockable Depends.	Hardcoded to single microservice scenario; needed parameterization.

3. Detailed Run Analysis & Four-Note Evaluations

Run 0: Baseline (The Lazy Prompt)

Added Layer: None (Unconstrained Baseline)

Prompt:

Write backend code

Representative Output Excerpt:

```
// Node.js / Express
const express = require('express');
const app = express();
let items = [{ id: 1, name: 'Item One' }];
app.get('/api/items', (req, res) => res.json(items));
app.post('/api/items', (req, res) => { items.push(req.body); res.status(201).json(req.body); });
app.listen(3000);

// Followed by definitions of GET/POST and 10 clarifying questions...
```

What changed in the prompt:	Zero layers applied. A genuinely weak and unguided directive.
What actually improved in the output:	Baseline reference point; no improvements.
What still failed:	Guessed arbitrary language (Node.js), invented a toy in-memory array, explained basic HTTP verbs, and asked 10 scoping questions.
What I would try next:	Add a clearer goal targeting user authentication, password reset, and session tokens.

Run 1: Layer 1 — Clearer Goal

Added Layer: A clearer goal

Prompt:

Write backend code to implement user authentication, password reset with secure tokens, and session token generation for a SaaS application.

Representative Output Excerpt:

```
from flask import Flask, request, jsonify
import jwt, sqlite3

@app.route('/login', methods=['POST'])
def login():
# queries sqlite3 auth.db, verifies password, encodes HS256 JWT
token = jwt.encode({'user_id': user[0], 'exp': ...}, 'secret')
return jsonify({'access_token': token})
```

What changed in the prompt:	Specified the domain goal: user authentication, password reset, and session token issuance for SaaS.
What actually improved in the output:	The model completely ceased generating toy item CRUD arrays and dropped textbook explanations. Output focused 100% on authentication mechanisms.
What still failed:	Assumed a synchronous Flask stack with direct unpooled SQLite queries (auth.db) and raw SQL strings, unsuitable for high-concurrency SaaS.

What I would try next:	Add real context specifying Python 3.11, FastAPI, async SQLAlchemy 2.0 (asyncpg), and Redis 7.0.

Run 2: Layer 2 — Real Context

Added Layer: Real context

Prompt:

Write backend code to implement user authentication, password reset with secure tokens, and session token generation for a SaaS application.

Context: The service is built with Python 3.11 and FastAPI, utilizing PostgreSQL via async SQLAlchemy 2.0 (asyncpg) for persistence and Redis 7.0 for session revocation caching and rate limiting.

Representative Output Excerpt:

```
from fastapi import FastAPI, Depends
from sqlalchemy.ext.asyncio import AsyncSession
import redis.asyncio as aioredis

# [... 350 lines combining models, schemas, password hashing,
# routes, and Redis revocation into a single monolithic file ...]
```

What changed in the prompt:	Injected production environment: Python 3.11, FastAPI, asyncpg, DeclarativeBase ORM, and Redis 7.0.
What actually improved in the output:	Eliminated synchronous blocking code. Instantly adopted async def endpoints, AsyncSession dependencies, SQLAlchemy Mapped types, and Redis async revocation.
What still failed:	Produced a 350-line monolithic blob where database models, Pydantic schemas, crypto helpers, and endpoints were smashed together without modular separation.
What I would try next:	Add constraints to force brevity and avoid heavy third-party dependencies.

Run 3: Layer 3 — Constraints (Honest Regression / 'This Made It Worse')

Added Layer: Constraints

Prompt:

Write backend code to implement user authentication, password reset with secure tokens, and session token generation for a SaaS application.

Context: The service is built with Python 3.11 and FastAPI, utilizing PostgreSQL via async SQLAlchemy 2.0 (asyncpg) for persistence and Redis 7.0 for session revocation caching and rate limiting.

Constraints: Do not use any third-party auth packages or cryptographic libraries (like passlib, bcrypt, or python-jose); rely only on Python standard libraries; keep the total response under 60 lines of code; do not include comments or docstrings.

Representative Output Excerpt:

```
def hash_pw(pw: str) -> str:
return hashlib.sha256(pw.encode()).hexdigest() # CRITICAL VULNERABILITY: UNSALTED!

def make_token(uid: str) -> str:
# Insecure base64 + hmac token without constant-time comparison
```

```
sig = hmac.new(SECRET.encode(), payload.encode(), hashlib.sha256).hexdigest()
return f"{payload}.{sig}"

@app.post("/login")
async def login(req: Request):
if b.get("pw") == "admin": return {"tok": make_token(b["user"])} # HARDCODED!
```

HONEST REGRESSION IDENTIFIED: Imposing arbitrary negative constraints (<60 lines, standard libraries only) caused a critical failure mode where the model traded security compliance and architectural integrity for superficial brevity.

What changed in the prompt:	Imposed blunt negative constraints: standard libraries only, under 60 lines, and no comments.
What actually improved in the output:	Output compressed to 40 lines; eliminated rambling explanations.
What still failed (Honest 'Made it worse'):	SEVERE QUALITY & SECURITY BREAKDOWN: Unsalted SHA-256 (vulnerable to rainbow tables), homebrew tokens vulnerable to timing attacks (used != instead of compare_digest), deleted all PostgreSQL and Redis code, and hardcoded 'admin' credentials. Negative brevity constraints broke production viability.
What I would try next:	Abandon negative compression constraints. Replace them with a specified output format enforcing clean modular files.

Run 4: Layer 4 — Specified Output Format

Added Layer: A specified output format

Prompt:

Write backend code to implement user authentication, password reset with secure tokens, and session token generation for a SaaS application.

Context: The service is built with Python 3.11 and FastAPI, utilizing PostgreSQL via async SQLAlchemy 2.0 (asyncpg) for persistence and Redis 7.0 for session revocation caching and rate limiting.

Specified output format: Present the solution as three distinct, production-ready modules formatted in separate markdown code blocks with explicit file headers:
1. `app/schemas/auth.py`: Pydantic v2 schemas
2. `app/core/security.py`: Password hashing (passlib[bcrypt]) and JWT (python-jose)
3. `app/api/v1/auth.py`: FastAPI APIRouter endpoints with dependency injection.

Representative Output Excerpt:

```
### app/schemas/auth.py
class UserRegisterRequest(BaseModel): ...

### app/core/security.py
pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")
def create_access_token(...) -> tuple[str, str]: ...

### app/api/v1/auth.py
@router.post("/login", response_model=TokenResponse)
async def login(payload: UserLoginRequest, db: AsyncSession = Depends(get_db)): ...
```

What changed in the prompt:	Replaced negative brevity rules with a 3-tier modular file layout (schemas, security core, and API routes).

What actually improved in the output:	Healed the Run 3 regression while resolving Run 2's monolithic clutter. Created clean, pasteable files with proper separation of concerns matching enterprise standards.
What still failed:	Zero automated verification. There is no proof that token expiration or Redis blocklisting works as expected under real test vectors.
What I would try next:	Add verification requirements mandating an async pytest suite testing success, 401 errors, expired tokens, and Redis revocation.

Run 5: Layer 5 — Verification Requirements

Added Layer: Verification requirements

Prompt:

[Prior prompt from Run 4] +
Verification requirements: Provide a fourth module `tests/test_auth.py` containing an asynchronous pytest suite (using pytest-asyncio and httpx.AsyncClient) asserting: 1) register & login JWT issuance, 2) invalid password returns HTTP 401, 3) expired JWT returns 401, 4) logout revokes token in Redis.

Representative Output Excerpt:

```
# tests/test_auth.py
@pytest.mark.asyncio
async def test_auth_full_lifecycle_success(mock_redis):
app.dependency_overrides[get_redis] = lambda: mock_redis
async with AsyncClient(transport=ASGITransport(app=app), base_url="http://test") as client:
# Tests register, login, protected endpoint, logout, and token revocation block
```

What changed in the prompt:	Added verification requirements specifying an automated async pytest suite testing 4 explicit test vectors.
What actually improved in the output:	Acted as an architectural forcing function. To make the test suite work with mocks, the model refactored the router to expose clean Depends(get_db) and Depends(get_redis) injection, dramatically increasing testability.
What still failed:	The prompt was hardcoded to this single auth microservice; it was not yet parameterized for other backend modules.
What I would try next:	Generalize into a clean, reusable template with parameter brackets for any backend engineer.

4. Final Reusable Prompt Template (Stranger-Ready)

Through empirical iterations, the prompt was transformed from an embarrassing 3-word query into a battle-tested engineering template. It is packaged with parameterized variable slots so any backend engineer on the team can achieve production-grade code on the first generation without author supervision:

```
Role: Senior Backend Infrastructure Engineer
Objective: Implement [FEATURE_NAME] for our [DOMAIN_TYPE] backend service.

Context & Environment:
- Language & Runtime: Python 3.11+
- Framework: FastAPI (modern asynchronous patterns, dependency injection)
- Persistence: PostgreSQL via SQLAlchemy 2.0 (asyncpg, DeclarativeBase, Mapped types)
- Cache & Session Store: Redis 7.0 (redis.asyncio)
- Secret Management: Configuration via pydantic-settings BaseSettings reading env vars

Requirements & Business Logic:
1. [REQUIREMENT_1 - e.g., Email validation and salted bcrypt password hashing]
2. [REQUIREMENT_2 - e.g., JWT access token issuance (15m exp) and rotation]
3. [REQUIREMENT_3 - e.g., Token revocation endpoint on logout that blacklists JTI in Redis]

Specified Output Format:
Organize the implementation across clean, modular files with explicit markdown file headers:
1. `app/schemas/[MODULE].py`: Strict Pydantic v2 schemas for Request/Response models.
2. `app/core/[UTILITY].py`: Reusable domain logic, security routines, and hashing helpers.
3. `app/api/v1/[MODULE].py`: FastAPI APIRouter endpoints using Depends for DB and cache.
4. `tests/test_[MODULE].py`: Asynchronous test suite using pytest-asyncio and httpx.AsyncClient.

Verification & Quality Criteria:
- Do not omit implementation details with '# TODO: implement later' placeholders.
- Ensure all route exceptions raise typed HTTPException with RFC-7807 compliant error details.
- All database operations must commit or rollback cleanly within async session contexts.
- The test suite in tests/test_[MODULE].py must assert happy paths, 401 errors, and revocation.
```

5. Project Artifacts & GitHub Verification

All 6 progression runs, comparative analyses, and the final stranger-ready template have been committed and pushed to GitHub:

GitHub Repository:	https://github.com/deepak007679/prompt-ladder-assignment
Local Directory:	C:\Users\Deepak\.gemini\antigravity\scratch\prompt-ladder-assignment
Primary Deliverable:	PROMPT_LADDER.md (Complete 6-run ladder documentation)
Reusable Template:	final_reusable_prompt.md (Production prompt template)
Individual Run Logs:	runs/00_baseline.md through runs/05_version_5_verification.md
Git Commit:	9b978b0 (feat: complete prompt ladder assignment with 6 runs and reusable prompt)